have a separate virtual environment for each project you create. This avoids any library conflicts and means you can switch back and forth between projects without worrying about whether they'll still work.

There are several virtual environment management tools out there. In this book, all the examples use `venv`, which ships with Python. To create a virtual environment using `venv`, type the following:

```
$ python -m venv my_environment
```

> **NOTE** If you have multiple versions of Python installed on your system, make sure to use the version that you want to use inside the virtual environment.

This example creates a directory named my_environment, containing the virtual environment. Common practice is to name the folder **venv** or **.venv**. With the directory created, you activate it. On Windows, run the following:

```
C:> my_environment/Scripts/activate.bat
```

On macOS and Linux, use

```
$ source my_environment/bin/activate
```

You can switch to a different environment by activating it instead. If you wish to stop using the environment, you use the deactivate command, which is the same on all operating systems:

```
$ deactivate
```

For more information on `venv`, see the documentation: https://docs.python.org/3/tutorial/venv.html.

Virtual environment directory location

The `venv` tool gets used to create the directory containing the virtual environment in the folder location where it is run. There are two schools of thought on the placement of this directory: (1) in your project or (2) in a central location.

Keeping your virtual environment with your project makes it clear what environment goes with the project and keeps things all together. There are even tools out there that can automatically switch your virtual environment based on what folder you are in. There are two downsides to this approach: you can't easily share virtual environments, and it complicates the use of some command line tools. Sharing virtual environments isn't too important, but it can be handy if you have multiple pieces of software that will all be using the same environment in production.

Depending on how often you use command-line tools, virtual environments inside a project folder can add extra complications. I often run a Unix `find` command to look for files or `grep -r` to look for content in files. By having the virtual environment directory in the project, you end up searching all those files as well. As most Python third-party libraries contain Python, searching for or in files ending in .py tends to return a lot of false positives.